Mathematical Reference

Contents

Mathematical Reference and Algorithm Documentation

This document provides a detailed mathematical description of the algorithms used in the Peak Analysis Tool. It is intended for researchers and developers who wish to understand the underlying signal processing and physical corrections implemented in the software.

1. Peak Detection

The core peak detection relies on identifying local maxima that satisfy specific criteria regarding prominence, width, and distance.

1.1 Detection Algorithm

The peak detection is primarily based on the prominence of peaks. A peak is defined as a local maximum x_p at index i_p such that:

$$x_{i_p} \geq x_i \quad \forall i \in [i_p - w, i_p + w]$$

where w is the window size.

The algorithm uses `scipy.signal.find_peaks` with enhanced pre- and post-processing.

1.2 Prominence and Filtering

Peak prominence (P) measures how much a peak stands out from the surrounding baseline. It is defined as the vertical distance between the peak and its lowest contour line.

$$P_i = h_i - \max(\min(h_{left}, h_{right}))$$

where h_i is the peak height, and h_{left}, h_{right} are the heights of the nearest higher peaks or signal boundaries.

Subpeak Filtering: To distinguish main signal peaks from noise or substructures, we implement a **Prominence Ratio** filter. A peak is discarded if its prominence is small relative to its absolute height:

$$R_{prom} = \frac{P_i}{H_i} < \tau_{ratio}$$

where H_i is the absolute height of the peak and τ_{ratio} is the configurable threshold (default: 0.5). This is crucial for filtering out noise riding on top of larger signal variations.

1.3 Peak Width Estimation

Peak widths are calculated at a relative height (default 50%, FWHM). For a peak at i_p with height H and baseline B, the width is measuring at level L:

$$L = H - \text{rel\_height} \times (H - B)$$

Linear interpolation is used to find the exact intersection points with the signal curve to provide sub-sample accuracy.

2. Signal Processing

2.1 Butterworth Low-Pass Filter

To reduce high-frequency noise, we apply a digital Butterworth filter. The transfer function of an N-th order Butterworth low-pass filter is given by:

$$|H(j\omega)|^2 = \frac{1}{1 + (\frac{\omega}{\omega_c})^{2N}}$$

where ω_c is the cutoff frequency.

Adaptive Cutoff Frequency: The software can automatically determine an optimal cutoff frequency ω_c based on the signal content. It estimates the average width ($\bar{W}$) of the narrowest 10% of peaks (representing the fastest signal features) and sets:

$$f_c = \frac{1}{\bar{W}} \times \text{normalization\_factor}$$

This ensures the filter preserves the shape of the sharpest real peaks while attenuating faster noise.

2.2 Savitzky-Golay Filter

As an alternative smoothing method, a Savitzky-Golay filter can be applied. This performs a local polynomial regression to smooth the data. For a window of length $2M + 1$ and polynomial order k, the smoothed point y_j is:

$$y_j = \sum_{i=-M}^{M} C_i x_{j+i}$$

where C_i are convolution coefficients derived from least-squares fitting of the polynomial. The window length is automatically estimated as 1.5× the average peak width if not specified.

3. Physical Corrections

3.1 Dead-Time Correction

Photon counting detectors often exhibit a "dead time" (τ_D) after registering a photon, during which they are blind to subsequent events. This leads to nonlinearity at high count rates. We correct for this using the non-paralyzable model.

Let: * $R_{measured}$: The measured count rate (counts/second). * τ_D: The detector dead time (seconds, typically ~43 ns for our detectors). * R_{true}: The actual incident photon rate.

The relationship is:

$$R_{measured} = \frac{R_{true}}{1 + R_{true}\tau_D}$$

Inverting this to solve for the true rate:

$$R_{true} = \frac{R_{measured}}{1 - R_{measured}\tau_D}$$

The correction factor F_{corr} applied to the raw signal counts C_{raw} is:

$$C_{corr} = C_{raw} \times \frac{1}{1 - R_{measured}\tau_D}$$

Singularity Handling: The correction approaches infinity as $R_{measured} \to 1/\tau_D$ (saturation). We clamp the correction factor to a maximum (default 10x) to prevent numerical instability near saturation:

$$F_{corr} = \min\left(\frac{1}{1 - R_{measured}\tau_D}, F_{max}\right)$$

4. Analysis Metrics

4.1 Peak Area (Integration)

The total photon count for a burst (Area) is calculated using the trapezoidal rule over the peak window $[a, b]$:

$$Area = \int_a^b (S(t) - B)dt \approx \sum_{i=a}^{b-1} \frac{(S_i - B) + (S_{i+1} - B)}{2}\Delta t$$

where $S(t)$ is the signal, B is the local background level (minimum value in the window), and Δt is the sampling interval.

4.2 Signal-to-Noise Ratio (SNR)

The SNR for each peak is calculated relative to the baseline noise:

$$\text{SNR} = \frac{H_{peak} - \mu_{baseline}}{\sigma_{baseline}}$$

where $\mu_{baseline}$ and $\sigma_{baseline}$ are the mean and standard deviation of the signal in non-peak regions. We use a Robust estimator for $\sigma_{baseline}$ based on the Median Absolute Deviation (MAD):

$$\sigma_{robust} \approx 1.4826 \times \text{median}(|x - \text{median}(x)|)$$

This prevents large peaks from inflating the noise estimate.

References

1. Virtanen, P. et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". *Nature Methods*, 2020.
2. Savitzky, A., & Golay, M. J. E. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures". *Analytical Chemistry*, 1964.